Unsorted Array

Example Usage in `main()`

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_SIZE 100

int main() {

    int arr[MAX_SIZE], size = 0;

    insert(arr, &size, 5);  insert(arr, &size, 2);
    insert(arr, &size, 8);  insert(arr, &size, 1);
    printf("Array: ");
    for (int i = 0; i < size; i++) printf("%d ", arr[i]); printf("\n");

    int keyToSearch = 8;
    int searchResult = search(arr, size, keyToSearch);
    if (searchResult != -1)
        printf("%d found at index %d\n", keyToSearch, searchResult);
    else
        printf("%d not found\n", keyToSearch);
```

Unsorted Array

Example Usage in `main()`

```
deleteByPointer(arr, &size, 1);
printf("Array after deleting element at index 1: ");
for (int i = 0; i < size; i++) printf("%d ", arr[i]); printf("\n");

deleteByKey(arr, &size, 2);
printf("Array after deleting element with key 2: ");
for (int i = 0; i < size; i++) printf("%d ", arr[i]); printf("\n");

printf("Minimum element in the array: %d\n", findMin(arr, size));

deleteMin(arr, &size);
```

Unsorted Array

Example Usage in `main()`

```
    printf("Array after deleting the minimum element: ");
    for (int i = 0; i < size; i++) printf("%d ", arr[i]); printf("\n");

    decreaseKey(arr, size, 1, 0);
    printf("Array after decreasing the key at index 1 to 0: ");
    for (int i = 0; i < size; i++) printf("%d ", arr[i]); printf("\n");

    int array2[] = {3, 7, 4};
    meld(arr, &size, array2, sizeof(array2) / sizeof(array2[0]));
    printf("Array after melding with another array: ");
    for (int i = 0; i < size; i++) printf("%d ", arr[i]); printf("\n");

    return 0;
}
```

Sorted Array

Sorted Array

Introduction

- Array data structure where the elements are arranged either in ascending or in descending order.
- Searching for a particular element is efficient, as binary search can be employed.

Sorted Array

Operations: Overview

- **Search:** Binary search for finding specific elements.
- **Insert:** Shift the larger elements to the right by one position and insert the element.
- **Delete (specified with pointer):** Remove the specified element and potentially shift others.
- **Delete (specified with key):** Use binary search to locate the key and adjust the array.
- **FindMin:** The minimum element is always at the first index.
- **DeleteMin:** Remove the element at the first index and shift remaining elements.
- **DecreaseKey:** Delete the specified element and reinsert it with decreased key.
- **Meld:** Merge two sorted arrays into a single sorted array.

Sorted Array

Time bounds

If n is the current size of the data structure,

- **Search:** $O(\log n)$
- **Insert:** $O(n)$
- **Delete (specified with pointer):** $O(n)$.
- **Delete (specified with key):** $O(n)$.
- **FindMin:** $O(1)$.
- **DeleteMin:** $O(n)$.
- **DecreaseKey:** $O(n)$.
- **Meld:** $O(n)$.

Sorted Array

Helper Function: binarySearch

```
// Binary search
// Function to search for an element in the sorted array
int search(int arr[], int size, int key) {
    int left = 0, right = size - 1;
    while (left <= right) {
        int mid = (left + right) / 2;
        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            left = mid + 1;
        else
            right = mid - 1;
    }
    return -1; // Element not found
}
```

The search space halves in every step. $T(n) = T\left(\frac{n}{2}\right) + 1 = O(\log n)$.

Sorted Array

Implementation of Insert

```
// Function to insert an element into the sorted array
void insert(int arr[], int *psize, int key) {
    if (*psize >= MAX_SIZE) {
        printf("Error: Array is full\n");
        return;
    }

    int i = *psize - 1;
    while (i >= 0 && arr[i] > key) {
        arr[i + 1] = arr[i];
        i--;
    }
    arr[i + 1] = key;
    (*psize)++;
}
```

Sorted Array

Implementation of deleteByPointer

```
// Function to delete an element specified with a pointer
void deleteByPointer(int arr[], int *psize, int index) {
    if (index < 0 || index >= *psize) {
        printf("Error: Invalid pointer\n");
        return;
    }

    while (index < *psize - 1) {
        arr[index] = arr[index + 1];
        index++;
    }
    (*psize)--;
}
```

Sorted Array

Implementation of deleteByKey

```
// Function to delete an element specified with a key
void deleteByKey(int arr[], int *psize, int key) {
    int index = search(arr, *psize, key);
    if (index != -1)
        deleteByPointer(arr, psize, index);
    else
        printf("Error: Element not found\n");
}
```

Sorted Array

Implementation of findMin

```
// Function to find the minimum element in the sorted array
int findMin(int arr[], int size) {
    if (size == 0) {
        printf("Error: Array is empty\n");
        return -1;
    }
    return arr[0];
}
```

Sorted Array

Implementation of deleteMin

```
// Function to delete the minimum element in the sorted array
void deleteMin(int arr[], int *psize) {
    if (*psize == 0) {
        printf("Error: Array is empty\n");
        return;
    }
    deleteByPointer(arr, psize, 0);
}
```

Sorted Array

Implementation of decreaseKey

```
// Function to decrease the key of an element at a specified index
void decreaseKey(int arr[], int *psize, int index, int newValue) {
    if (index >= 0 && index < size) {
        if (arr[index] > newValue) {
            deleteByPointer(arr, psize, index);
            insert(arr, psize, newValue);
        }
    } else {
        printf("Error: Invalid index\n");
    }
}
```

Sorted Array

Implementation of meld

```
void meld(int arr1[], int *psize1, int arr2[], int size2) {
    if (*psize1 + size2 <= MAX_SIZE) {
        for (int i = 0; i < size2; i++)
            arr1[*psize1 + i] = arr2[i];
        merge(arr, 0, *psize1 - 1, *psize1 + size2 - 1);
    } else
        printf("Error: Meld would exceed array size limit\n");
}
```

Queue

Queue

FIFO - First In, First Out

- Elements are added at the **rear** and removed from the **front**.
- It operates in the same way as a queue of people waiting for a service.
- Queue operations:
  - **Enqueue**: Add an element to the rear of the queue.
  - **Dequeue**: Remove an element from the front of the queue.
  - **isEmpty**: Check if the queue is empty.
  - **isFull**: Check if the queue is full (in case of a bounded queue).
  - **Front**: Get the element at the front without removing it.

Queue

Implementation of isEmpty and isFull

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_SIZE 100

int queue[MAX_SIZE];
int front = -1;  int rear = -1;

// Function to check if the queue is empty
int isEmpty() {
    return front == -1;
}

// Function to check if the queue is full
int isFull() {
    return (rear + 1) % MAX_SIZE == front;
}
```

Queue

Implementation of Front

```
// Function to return the element at the front of the queue
int Front() {
    if (isEmpty()) {
        printf("Error: Queue is empty\n");
        return -1;
    }
    return queue[front];
}
```

Queue

Implementation of Enqueue

```
// Function to enqueue an element
void enqueue(int value) {
    if (isFull()) {
        printf("Error: Queue is full\n");
        return;
    }

    if (isEmpty()) {
        front = 0;
    }

    rear = (rear + 1) % MAX_SIZE;
    queue[rear] = value;
}
```

Queue

Implementation of Dequeue

```
// Function to dequeue an element
void dequeue() {
    if (isEmpty()) {
        printf("Error: Queue is empty\n");
        return;
    }

    printf("Dequeued element: %d\n", queue[front]);
    if (front == rear) {
        // Reset queue when last element is dequeued
        front = rear = -1;
    } else {
        front = (front + 1) % MAX_SIZE;
    }
}
```